

Lab4 Report

22300240022 王镜凯

实验概述

本实验实现了从抽象语法树(AST)到中间表示(IR)的转换，主要涉及FDMJ语言到Tree IR的翻译

三种IR表达式

```
class Tr_ex { // 计算值的表达式
    tree::Exp* exp;
};

class Tr_nx { // 执行效果的语句
    tree::Stm* stm;
};

class Tr_cx { // 条件跳转
    Patch_list* true_list; // 真值跳转目标列表
    Patch_list* false_list; // 假值跳转目标列表
    tree::Stm* stm; // 条件判断语句
};
```

三种表达式可以通过以下方法相互转换：

- unEx(): 转换为计算值的表达式
- unNx(): 转换为执行效果的语句
- unCx(): 转换为条件跳转

条件语句转换

If

```
void ASTToTreeVisitor::visit(fdmj::If* node) {  
    // 1. 处理条件表达式  
    node->exp->accept(*this);  
    Tr_cx* cond_tr = dynamic_cast<Tr_cx*>(tr_exp);  
  
    // 2. 生成标签  
    tree::Label* then_label = temp_map->newlabel();  
    tree::Label* else_label = temp_map->newlabel();  
    tree::Label* end_label = temp_map->newlabel();  
  
    // 3. 打补丁，连接跳转目标  
    cond_tr->true_list->patch(then_label);  
    cond_tr->>false_list->patch(else_label);  
  
    // 4. 处理then和else分支  
    // ...  
}
```

While

```
void ASTToTreeVisitor::visit(fdmj::While* node) {  
    // 1. 生成循环相关标签  
    tree::Label* test_label = temp_map->newlabel();  
    tree::Label* body_label = temp_map->newlabel();  
    tree::Label* end_label = temp_map->newlabel();  
  
    // 2. 记录break/continue标签  
    while_labels.push_back({test_label, end_label});  
  
    // 3. 处理循环条件和循环体  
    // ...  
  
    // 4. 构建循环结构
```

```

vector<tree::Stm*>* sl = new vector<tree::Stm*>();
sl->push_back(new tree::LabelStm(test_label));
sl->push_back(cond_tr->stm);
sl->push_back(new tree::LabelStm(body_label));
sl->push_back(body_tr->stm);
sl->push_back(new tree::Jump(test_label));
sl->push_back(new tree::LabelStm(end_label));
}

```

|| &&

```

// 逻辑或(||)的实现
if (op == "||") {
    // 左操作数为假时才计算右操作数
    tree::Label* second_label = temp_map->newlabel();
    left_cx->>false_list->patch(second_label);

    // 合并true列表（任一为真即为真）
    left_cx->>true_list->add(right_cx->>true_list);
}

// 逻辑与(&&)的实现
if (op == "&&") {
    // 左操作数为真时才计算右操作数
    tree::Label* second_label = temp_map->newlabel();
    left_cx->>true_list->patch(second_label);

    // 合并false列表（任一为假即为假）
    left_cx->>false_list->add(right_cx->>false_list);
}

```

技术难点

1. 条件表达式的处理

- 难点：需要正确处理短路逻辑
- 解决：使用true_list和false_list进行延迟绑定

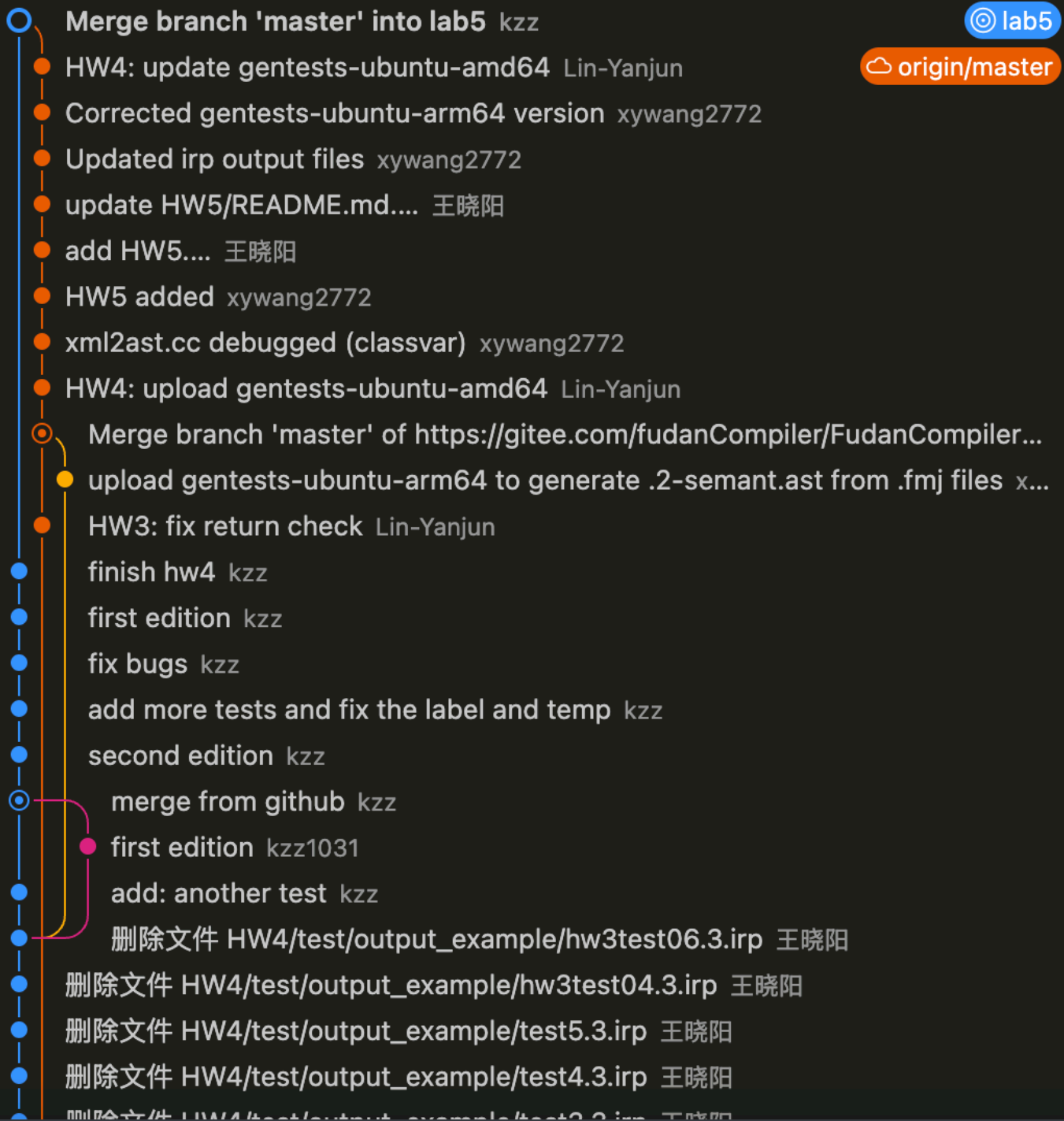
2. break/continue实现

- 难点：需要找到对应的循环跳转目标
- 解决：维护while_labels栈存储当前循环的标签信息

3. Tr_exp

- 一开始用visit_tree_result返回递归结果，导致代码比较冗长，后来将所有的返回值都存成Tr_exp形式，保持了输出与example的一致

Git Graph



参考资料

课程ppt, 虎书